

TOPIC 01: OOPS IN JS & OBJECT PROTOTYPES

OOPs Ka Asli Purpose:

Object-Oriented Programming se hum apne code ko real-world entities ki tarah model karte hain. Data (**Properties**) aur kaam (**Methods**) ek sath pack rehte hain, jisse code clean aur reusable banta hai (DRY Principle).

OOPs Ke 4 Main Pillars:

1. ENCAPSULATION

2. INHERITANCE

3. POLYMORPHISM

4. ABSTRACTION

Object Prototypes & Chaining:

JavaScript mein inheritance **prototype-based** hoti hai. Har JS object ke paas ek hidden pointer hota hai jisko `[[Prototype]]` ya `__proto__` bolte hain. Agar koi method object pe na mile, toh JS uske prototype mein dhoondhta hai.

MYARR

→ `__proto__` →

ARRAY.PROTOTYPE

→ `__proto__` →

OBJECT.PROTOTYPE

→ `__proto__` →

NULL



JS prototypes.js

```
1  const arr = [1, 2, 3];
2  console.log(arr.__proto__ === Array.prototype);           // true (Array ke methods mil gaye)
3  console.log(Array.prototype.__proto__ === Object.prototype); // true (Object level tak pahuncha)
4  console.log(Object.prototype.__proto__);                  // null (chain yahan khatam hoti hai)
5
```

TOPIC 02: FACTORY FUNCTIONS VS CONSTRUCTORS & NEW

Factory Functions (Kyun Fail Huye?):

Factory function har baar naya object banata hai, jisse har instance ke liye methods memory mein alag-alag copy hote hain (Heavy Memory Waste!).

Constructor Functions & 'new' Keyword:

Constructor ek blueprint hota hai. Jab hum **new** lagate hain, toh JS background mein 4 kaam karta hai:

- 1. Ek brand new empty object `{}` banta hai.
- 2. `this` ko usi naye object se bind karta hai.
- 3. Naye object ke `__proto__` ko `Constructor.prototype` se jodta hai.
- 4. Naye object ko automatically return karta hai.

```
1 function Person(name, age) {
2   this.name = name;
3   this.age = age;
4 }
5 // Prototype par method attach karne se memory bachti hai
6 Person.prototype.talk = function() {
7   console.log(`Namaste, main hoon ${this.name}!`);
8 };
9 const p1 = new Person("Shivam", 22);
10 p1.talk(); // Output: Namaste, main hoon Shivam!
```

TOPIC 03: MODERN ES6 CLASSES SYNTAX

Class: Prototypes Ka Modern Wrapper:

ES6 mein class introduce hua. Ye Java/C++ jaisa clean syntax deta hai, par andar se 100% prototype-based hi kaam karta hai (Syntactical Sugar).

```
JS Person.js
1 class Person {
2   constructor(name, age) {
3     this.name = name;
4     this.age = age;
5   }
6   greet() {
7     return `Hello, main hoon ${this.name}, umar: ${this.age}`;
8   }
9 }
10 const user = new Person("Shivam", 22);
```

TOPIC 04: STATIC METHODS & STATIC PROPERTIES

Class-Level Members (Instances Se Call Nahi Hote!):

static keyword wale methods aur properties direct class ke hote hain, kisi individual object ke nahi. Unko bina new ke direct `ClassName.method()` karke call kiya jata hai.

- **Use Case:** Utility functions (jaise `Math.floor()`), database connections, data converters.

```
JS MathUtil.js
1 class MathUtil {
2   static PI = 3.14159; // Static property
3
4   static calculateCircleArea(radius) { // Static helper method
5     return MathUtil.PI * radius * radius;
6   }
7 }
8 console.log(MathUtil.PI); // 3.14159 (Direct class se)
9 console.log(MathUtil.calculateCircleArea(10)); // 314.159
10 const m = new MathUtil();
11 // m.calculateCircleArea(10); // TypeError! Instance ke paas ye method nahi hota!
```

TOPIC 05: GETTERS & SETTERS (GET & SET)

Validation Ke Sath Data Control:

Getters aur Setters se hum method ko property ki tarah read/write kar sakte hain, par peechhe hum validation aur security check chala sakte hain.

```
1  class Temperature {
2      constructor(celsius) { this.celsius = celsius; }
3
4      get fahrenheit() { // Getter: property ki tarah read hota hai
5          return (this.celsius * 9/5) + 32;
6      }
7      set celsius(val) { // Setter: pehle check karega fir value assign karega
8          if (val < -273.15) throw new Error("Absolute zero se niche impossible hai!");
9          this._celsius = val;
10     }
11 }
12 const t = new Temperature(25);
13 console.log(t.fahrenheit); // 77 (Bina parenthesis call hua!)
```

TOPIC 06: PILLAR 1 – ENCAPSULATION

Encapsulation Kya Hai?

Encapsulation ka matlab hai data ko wrap karna aur external access se protect karna. Bahar wale code ko directly sensitive data chhoone ki permission nahi hoti, sirf safe public methods ke through interact kar sakte hain.

Modern ES2022 Private Fields (#) & Private Methods:

Variable ke aage # lagane se wo strictly private ban jata hai. Class ke bahar se access karne par syntax error aata hai!

```
JS BankAccount.js
1  class BankAccount {
2      owner;           // Public field (koi bhi dekh sakta hai)
3      #balance = 0;    // Private field (bahar se invisible!)
4      #pin;            // Private field (security pin)
5
6      constructor(owner, initialDeposit, pin) {
7          this.owner = owner;
8          this.#balance = initialDeposit;
9          this.#pin = pin;
10     }
11     // Private Method: Sirf class ke andar chalega
12     #checkPin(enteredPin) { return this.#pin === enteredPin; }
13
14     withdraw(amount, pin) {
15         if (!this.#checkPin(pin)) throw new Error("Galat PIN!");
16         if (amount > this.#balance) throw new Error("Insufficient funds!");
17         this.#balance -= amount;
18         return `₹${amount} nikal gaye. Bacha balance: ₹${this.#balance}`;
19     }
20     get balance() { return this.#balance; } // Read-only view
21 }
22 const khata = new BankAccount("Shivam", 5000, 1234);
23 // khata.#balance = 999999; // SyntaxError: Private field bahar nahi touch ho sakta!
```

TOPIC 07: PILLAR 2 – INHERITANCE

Inheritance (Code Reusability):

Inheritance se ek Child class apne Parent class ke features aur methods ko inherit karti hai using extends keyword.

- `super()` → Parent class ke constructor ko call karta hai (Child mein `this` use karne se pehle ise call karna mandatory hai!).
- `super.method()` → Parent class ke method ko directly invoke karta hai.

```
JS Inheritance.js
1  class Person {
2      constructor(name, age) { this.name = name; this.age = age; }
3      introduce() { return `Namaste, main hoon ${this.name}.`; }
4  }
5
6  class Student extends Person {
7      constructor(name, age, marks) {
8          super(name, age); // Parent (Person) ka constructor chala
9          this.marks = marks;
10     }
11     padhai() { return `${this.name} exam ki taiyari kar raha hai.`; }
12 }
13 const s1 = new Student("Rahul", 20, 95);
14 console.log(s1.introduce()); // Person se inherit ho gaya!
15
16
```

TOPIC 08: PILLAR 3 – POLYMORPHISM

Polymorphism (Ek Naam, Anek Kaam):

Poly = Many, Morph = Forms . Jab alag-alag child classes ek hi naam ke method ko apne mutabik alag tarike se implement karein, toh usko Method Overriding bolte hain.

Dynamic Dispatch & Duck Typing:

JavaScript dynamic language hai ("Duck Typing: Agar ye duck ki tarah chalti hai aur bolti hai toh ye duck hai"). Ek hi loop mein alag-alag objects ka same method uniformly execute ho jata hai.

```
JS Polymorphism.js
1  class Animal {
2      speak() { return "Generic animal sound"; }
3  }
4
5  class Dog extends Animal {
6      speak() { return "Bho! Bho!"; }    // Method Overriding
7  }
8
9  class Cat extends Animal {
10     speak() { return "Meow! Meow!"; } // Method Overriding
11 }
12
13 // Polymorphism: Ek hi loop mein sabka apna speak() execute hoga
14 const animals = [new Dog(), new Cat(), new Animal()];
15 animals.forEach(a => {
16     console.log(a.speak()); // Bho! Bho! → Meow! Meow! → Generic animal sound
17 });
18
19
```

TOPIC 09: PILLAR 4 – ABSTRACTION

Abstraction: Complexity Ko Chhupana:

Abstraction ka matlab hai andar ki complex details ko chhupa kar user ko sirf ek simple aur clean interface dena (Jaise gaadi chhalate waqt sirf accelerator dabate hain, engine ke andar petrol kaise jal raha hai wo user se chhupa rehta hai).

JavaScript Mein Abstract Class Kaise Banayein?

JS mein direct abstract keyword nahi hota, par hum `new.target` check karke abstract class pattern banate hain taaki koi use direct instantiate na kar sake.

```
JS AbstractShape.js
1  class Shape {
2      constructor() {
3          if (new.target === Shape) {
4              throw new Error("Shape abstract class hai! Direct object nahi banega!");
5          }
6      }
7      // Abstract method contract: Har child class ko ise banana hi padega
8      calculateArea() {
9          throw new Error("calculateArea() method child class mein implement karo!");
10     }
11 }
12 class Circle extends Shape {
13     constructor(radius) { super(); this.radius = radius; }
14     calculateArea() { return Math.PI * this.radius * this.radius; }
15 }
16 // const s = new Shape(); // Error: Shape abstract class hai!
17 const c = new Circle(5);
18 console.log(c.calculateArea()); // 78.539
19
20
```

TOPIC 10: 'INSTANCEOF' OPERATOR

Prototype Chain Check Karna:

instanceof check karta hai ki kya constructor ka prototype us object ki poori prototype chain mein kahin bhi exist karta hai ya nahi.

```
JS instanceof.js
1 class Employee {}
2 class Manager extends Employee {}
3 const m = new Manager();
4
5 console.log(m instanceof Manager); // true
6 console.log(m instanceof Employee); // true (Parent se link hai!)
7 console.log(m instanceof Object); // true (Root parent hai!)
```

TOPIC 11: COMPOSITION VS INHERITANCE

Design Principle:

"Favor object composition over class inheritance."

- **Inheritance ("Is-a"):** Dog is an Animal. Hierarchical aur rigid hota hai; agar parent mein kuch badla toh saare child toot sakte hain.
- **Composition ("Has-a"):** Robot has a Flyer and Cleaner. Flexible hota hai; chote-chote features ko combine karke bada object banate hain.

TOPIC 12: 4 PILLARS MASTER SUMMARY TABLE

Pillar	Asli Matlab	JavaScript Mein Kaise Hota Hai?	Main Fayda
1. Encapsulation	Data hiding & security	#privateField, #privateMethod(), get/set	Sensitive data bahar se manipulate nahi ho sakta
2. Inheritance	Code reusability	class Child extends Parent, super()	Baar-baar same code nahi likhna padta (DRY)
3. Polymorphism	Ek naam ke multiple forms	Method overriding & dynamic dispatch	Uniform code loops aur easy feature extensions
4. Abstraction	Complexity chhupana	Public APIs, new.target === AbstractClass	Code use karna simple banta hai



Interview Pro-Tip: JavaScript mein private fields (#) runtime par V8 engine ke level par enforce hote hain. TypeScript ka private sirf compile-time par hota hai, jabki JS ka # truly hard-private hota hai!